



hel_py

Compilador Inspirado em Python

Arthur Silva · Gabriel Soares · João Victor · Lucas Enthony · Miqueias Barros · Patrick Costa

Visão Geral

Desenvolvimento e Objetivos do Sistema

| Objetivos & Tecnologias



Linguagem C#

Uso do framework .NET e Windows Forms para criação de uma interface robusta e funcional.



Fins Didáticos

Demonstrar conceitos de análise léxica, sintática e execução de código em sala de aula.



Mini IDE

Interface com editor de texto, console de saída, área de compilação e gestão de arquivos.

| Arquitetura do hel_py

Interface Gráfica

Interação com o usuário, comandos de arquivo (abrir/salvar) e visualização de saída.

Motor de Compilação

Leitura, validação, estruturação e execução lógica das instruções.

hel_py - demo.py

Escreva, salve e execute scripts Python dentro do hel_py.

Editor de código

Python subset

Console hel_py

```
1 x = 0
2 while x < 5:
3     print(x)
4     x = x + 1
5
6 if x == 5:
7     print("Fim do programa")
8 else:
9     print("Contador terminou com valor inesperado")
10
```

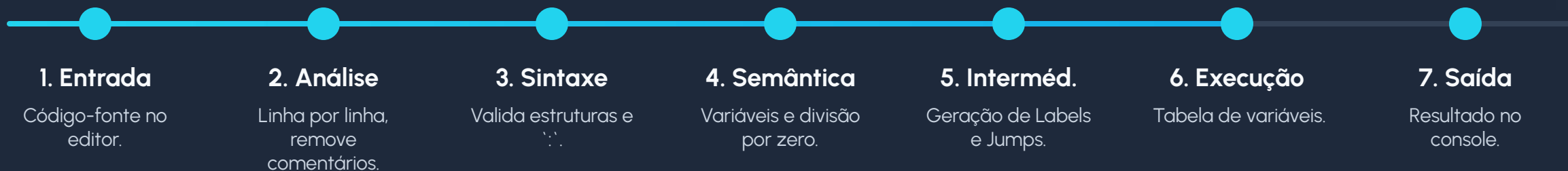
```
0
1
2
3
4
Fim do programa
[hel_py]: Programa finaliza
```

Código intermediário

```
Código intermediário gerado pelo hel_py
IO_PROGRAMA
x, 0
L while_1
ELSE x < 5 GOTO fim_while_2
T x
x, x + 1
while 1
```

Programa finalizada com sucesso.

| As 7 Etapas da Compilação



| Subconjunto Reconhecido



Expressões

Atribuições, cálculos aritméticos com precedência e comparadores lógicos.



Controle de Fluxo

Estruturas condicionais if/else e laços de repetição while.



Indentação

Validação de escopo baseada em recuo (padrão 4 espaços), assim como no Python.

| Equipe & Atividades (P1)



Arthur da Silva

Pesquisa inicial sobre compiladores, definição de escopo e seleção do subconjunto Python.



Gabriel Soares

Organização de testes funcionais, revisão de comparadores e análise de erros semânticos.



João Victor

Design da Interface Gráfica, usabilidade da IDE e integração do Windows Forms.

| Equipe & Atividades (P2)



Lucas Enthony

Criação dos algoritmos de teste (demo.py, combate_chefe.py) e validação da lógica de fluxos.



Miqueias Barros

Implementação do motor em C#, integração Front/Back, lógica de expressões e documentação.



Patrick Costa

Testes de qualidade (QA), validação dos requisitos do professor e tratamento de cenários de falha.

Tratamento de Erros



Léxico & Sintático

Comandos não reconhecidos e estrutura gramatical inválida.



Erros de Indentação

Identificação de blocos de código com espaçamento incorreto.



Variável não Declarada

Tentativa de uso de identificadores inexistentes na tabela de símbolos.



Divisão por Zero

Tratamento de exceções aritméticas durante o tempo de execução.

```
demo.py
execute scripts Python dentro do hel_py.

Python subset
Console hel_py
na linha 9: print("Contador terminou com valor inesperado")

5:
x)
+ 1

"Fim do programa")
"Contador terminou com valor inesperado"

liario
cao hel_py]: Comando nao reconhecido na linha 9: print("Contador terminou com valor inesperado")
```

Demonstração de Testes

Editor de código

Python subs

```
1 limite_inferior = 10
2 limite_superior = 15
3 numero_atual = limite_inferior
4 maior_sequencia = 0
5 numero_vencedor = 0
6
7 print("Calculando o problema de Collatz")
8 print("Testando de:")
9 print(limite_inferior)
10 print("Ate:")
11 print(limite_superior)
12
13 while numero_atual <= limite_superior:
14     passos = 0
15     n = numero_atual
16
17     while n > 1:
18         temp = n
19
20         while temp >= 2:
21             temp = temp - 2
22
23         if temp == 0:
24             dividendo = n
25             quociente = 0
26
```

Código intermediário

Arquivos de Exemplo

Validamos o sistema através de cenários reais:

 **demo.py** While, If, Else e incrementos.

 **if_else.py** Foco em estrutura condicional.

 **combate_chefe.py** Cenário complexo de RPG.

| Conclusão do Projeto



Impacto do hel_py

A construção do **hel_py** consolidou o conhecimento prático sobre o ciclo de vida de um compilador, unindo interface visual e lógica de execução em um ambiente acadêmico altamente produtivo.

PRINCIPAIS RESULTADOS

- ✓ Integração completa entre Front-end (C#) e Back-end (Lógica de Compilação).
- ✓ Validação de fluxos complexos e tratamento robusto de exceções.

| Encerramento

Obrigado pela atenção!



Repositório final no [GitHub](#)